SAEforestLearner: differences from SAEforest and the learner interface

Elia Vangi

2026-10-20

Contents

1. What is different from SAEforest 1
2. The learner interface 3
3. Use case: the eusilc example data with three learners 4
4. Choosing and tuning a learner with spatial cross-validation 10
5. Box-Cox transformation with any learner 11
References 12

1. What is different from SAEforest

`SAEforestLearner` is a fork of `SAEforest` 1.0.1 (Krennmair & Frink). The estimator of domain means is the mixed effects random forest (MERF) of Krennmair & Schmid (2022), with the same nonparametric bootstrap for the MSE; the original functions (`SAEforest_model()`, `MERFranger()`, `summarize_indicators()`, the generics) keep their names and arguments. The differences are in how the computation is organised and in what the fixed part of the model can be.

	SAEforest 1.0.1	SAEforestLearner
Population	a `data.frame` in memory	a `data.frame`, or a raster streamed from an on-disk cache (`SAEpopulation()`), tens of millions of units

	SAEforest 1.0.1	SAEforestLearner
Memory of the bootstrap	grows with population × `B` (dense matrices)	only domain sums are kept; bootstrap errors are drawn in C++ per replicate
Forest prediction	`ranger::predict` (sorts every covariate at each call)	flattened forests predicted in C++ (`flatten_forest()`), bit-identical
Output	domain means	means and totals (`total_factor`), with exact bootstrap MSEs of any aggregation (`SAEaggregate()`)
Transformation	none	optional Box-Cox MERF with empirical-best-type back-transformation (`transform = "boxcox"`)
Fixed part of the model	a `ranger` forest	**any supervised learner** (`learner =`): ranger (default), xgboost, lightgbm, glmnet, mgcv, lm, or user-defined (`sae_learner()`)
Out-of-sample predictions of the sample units	out-of-bag predictions of the forest	out-of-bag when the learner has them, otherwise K-fold cross-fitting (`oof_folds`)
Convergence of the MERF iterations	relative change of the log-likelihood	the same (`em_criterion = "original"`, default with ranger) or a robust criterion for stochastic learners (`"robust"`: windows, patience, best iteration kept)

	SAEforest 1.0.1	SAEforestLearner
Tuning	`tune_parameters()` (ranger, random CV)	`tune_learner()` + `spatial_folds()`: spatial-block CV of the mixed model for any learner
Reproducibility	`set.seed()`	`seed` argument; identical results for any number of cores

With `learner_ranger()` (the default) the point estimates are identical to those of the original package. The bootstrap random streams differ from version 2.0.0 of this package by two draws (see `NEWS`), so MSE estimates with a given seed are equal in distribution but not bit by bit.

The scalable engine is used whenever domain means (or totals) are requested from unit-level population data (`meanOnly = TRUE`, the default). Nonlinear indicators and aggregated covariate data still run the original code (`engine = "legacy"`), with the forest only.

2. The learner interface

A learner is an object of class `sae_learner` with two mandatory functions,

- `fit(x, y, args, threads)`: fits the learner to the covariates `x` (a data.frame, one row per sample unit) and the working target `y`; `args` is the list of learner parameters, `threads` the number of threads;
- `predict(model, x, threads)`: returns one prediction per row of `x` (a data.frame, or a numeric matrix for the population);

and optional ones: `oof(model, x, y, args, threads)` (out-of-sample predictions of the training units that the fit provides for free, e.g. out-of-bag predictions), `pack(model)` / `unpack(packed)` (a compact, serialisable form of the model for the bootstrap workers and the disk), `info(model)` (what `summary()` shows), plus `packages` (loaded in the workers), `args` (default parameters) and `bounded` (whether the predictions are bounded by the range of the training target, as for tree ensembles).

Two rules make a learner work in the parallel bootstrap: the functions must be **self-contained** (they are sent to the workers without their enclosing environment: call package functions as `pkg::fun` and keep the parameters in `args`), and the `...` of `SAEforest_model()` are passed to `fit()` through `args`, so ranger parameters such as `num.trees` must not be passed with other learners.

The MERF algorithm itself does not change: it alternates the learner, fitted to the target minus the current random effects, and a linear mixed model of the random intercepts with the learner's out-of-sample predictions as offset. When the learner has no out-of-sample predictions, they are obtained by K-fold cross-fitting at every iteration (and in the bias correction of the error variance), so that the random effects and the bootstrap residuals are never based on over-fitted in-sample predictions. With a linear learner (`learner_lm()`, exact leave-one-out predictions) the iterations converge to the nested-error linear mixed model, i.e. the EBLUP of the domain means.

3. Use case: the eusilc example data with three learners

The example data of `SAEforest` (synthetic EU-SILC data for Austria: 1,945 sample and 25,000 population units in 94 districts) are used with small settings (`B = 10`, 100 trees, 2 cores) so that the vignette runs in a few minutes. The target is equivalised income; the numeric covariates are used.

```
library(SAEforestLearner)
data("eusilcA_pop")
data("eusilcA_smp")
smp <- eusilcA_smp
X <- smp[, -c(1, 2, 16, 17, 18)]   # numeric covariates (gender dropped)
names(X)
#>  [1] "eqsize"      "cash"        "self_empl"   "unempl_ben"  "age_ben"
#>  [6] "surv_ben"    "sick_ben"    "dis_ben"     "rent"        "fam_allow"
#> [11] "house_allow" "cap_inv"     "tax_adj"
```

3.1 The default learner: ranger

```
t0 <- Sys.time()
m_rf <- SAEforest_model(Y = smp$eqIncome, X = X, dName = "district", smp_data = smp,
                        MSE = "nonparametric", B = 10, B_adj = 10, num.trees = 100, t
                        seed = 1, cores = 2, verbose = FALSE)
t_rf <- difftime(Sys.time(), t0, units = "secs")
m_rf$PopInfo$learner
#> [1] "ranger"
head(summarize_indicators(m_rf, MSE = TRUE, CV = TRUE)$ind, 4)
#>          district     Mean Mean_MSE    Mean_CV   Total   Total_MSE   Total_CV
#> 1      Amstetten 14302.46 232827.2 0.03373698 4591089 23990752238 0.03373698
#> 2          Baden 22823.55 207576.2 0.01996207 9083772 32880907510 0.01996207
#> 3        Bludenz 12530.18 703648.0 0.06694535 2029889 18466538117 0.06694535
#> 4 Braunau am Inn 12204.63 669073.7 0.06702123 3441706 53207413954 0.06702123
```

3.2 A predefined learner: gradient boosting (xgboost)

The parameters of the learner are given to its constructor; `oof_folds` sets the cross-fitting (5 by default); the robust convergence criterion is used automatically.

```
t0 <- Sys.time()
m_xgb <- SAEforest_model(Y = smp$eqIncome, X = X, dName = "district", smp_data = smp,
                         MSE = "nonparametric", B = 10, B_adj = 10, total_factor = 1,
                         verbose = FALSE, learner = learner_xgboost(nrounds = 150, et
                         oof_folds = 5)
t_xgb <- difftime(Sys.time(), t0, units = "secs")
m_xgb$PopInfo$learner
#> [1] "xgboost"
c(iterations = m_xgb$MERFmodel$IterationsUsed, best = m_xgb$MERFmodel$BestIteration)
#> iterations       best
#>          7          2
```

3.3 A user-defined learner: support vector regression (e1071)

A learner is defined with `sae_learner()`. The functions below are self-contained: `e1071::svm` is called with its namespace, the kernel parameters travel in `args`, and `predict` works both on the sample data.frame and on the population matrices. A radial kernel with scaling is used; `pack` keeps the model as is (an `svm` object is serialisable).

```
learner_svm <- function(cost = 1, gamma = NULL, epsilon = 0.1) {
  sae_learner(
    name = "svm",
    packages = "e1071",
    args = list(cost = cost, gamma = gamma, epsilon = epsilon),
    fit = function(x, y, args, threads) {
      m <- if (is.matrix(x)) x else data.matrix(x)
      args$importance <- NULL                        # a ranger argument added by the
      if (is.null(args$gamma)) args$gamma <- 1 / ncol(m)
      e1071::svm(x = m, y = y, type = "eps-regression", kernel = "radial", scale = TR
                 cost = args$cost, gamma = args$gamma, epsilon = args$epsilon)
    },
    predict = function(model, x, threads) {
      m <- if (is.matrix(x)) x else data.matrix(x)
      as.numeric(stats::predict(model, m))
    },
    info = function(model) list(type = "support vector regression (radial)", support_
  )
}
t0 <- Sys.time()
m_svm <- SAEforest_model(Y = smp$eqIncome, X = X, dName = "district", smp_data = smp,
                         MSE = "nonparametric", B = 10, B_adj = 10, total_factor = 1,
                         verbose = FALSE, learner = learner_svm(cost = 2), oof_folds
t_svm <- difftime(Sys.time(), t0, units = "secs")
summary(m_svm)
#> ------------------------------------------------------------------
```

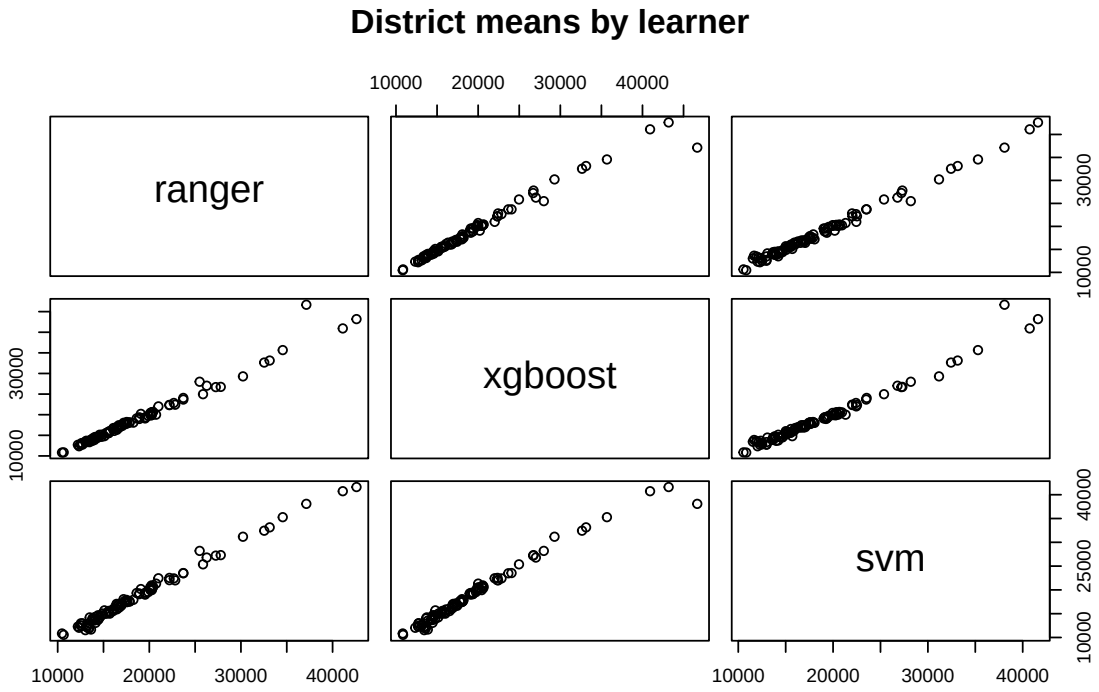
```
#> Mixed Effects Random Forest for Small Area Estimation
#> ------------------------------------------------------------
#> Call:
#> SAEforest_model(Y = smp$eqIncome, X = X, dName = "district",
#>     smp_data = smp, pop_data = eusilcA_pop, MSE = "nonparametric",
#>     B = 10, B_adj = 10, total_factor = 1, cores = 2, seed = 1,
#>     verbose = FALSE, learner = learner_svm(cost = 2), oof_folds = 5)
#>
#> Domains
#> ------------------------------------------------------------
#>  In-sample Out-of-sample Total
#>         70            24    94
#>
#> Totals:
#> Units in sample: 1945
#> Units in population: 25000
#>
#>                    Min. 1st Qu. Median      Mean 3rd Qu. Max.
#> Sample_domains       14    17.0   22.5  27.78571   29.00  200
#> Population_domains    5   126.5  181.5 265.95745  265.75 5857
#>
#> Random forest component:
#> ------------------------------------------------------------
#>
#> learner:                                        svm
#> type:            support vector regression (radial)
#> support_vectors:                                1388
#>
#> Structural component of random effects:
#> ------------------------------------------------------------
#> Linear mixed model fit by maximum likelihood  ['lmerMod']
#> Formula: Target ~ -1 + (1 | district)
```

```
#>    Data: data
#>  Offset: forest_preds
#>
#>      AIC      BIC   logLik deviance df.resid
#>  39195.5  39206.6 -19595.7  39191.5     1943
#>
#> Scaled residuals:
#>     Min      1Q  Median      3Q     Max
#> -4.1564 -0.4385 -0.0123  0.3405 12.4857
#>
#> Random effects:
#>  Groups   Name        Variance Std.Dev.
#>  district (Intercept)  6490328 2548
#>  Residual             30906746 5559
#> Number of obs: 1945, groups:  district, 70
#>
#> ICC:  0.1735518
#>
#> Convergence of MERF algorithm:
#> ----------------------------------------------------------------
#> Convergence achieved after 7 iterations. The fit of iteration 2 (highest log-lik
#> A maximum of 25 iterations used and tolerance set to: 1e-04
#>
#> Monitored Log-Likelihood:
#>  0 -19599.14 -19595.74 -19604.74 -19610.28 -19612.79 -19614.47 -19615.87
```

3.4 Comparing the three models

The point estimates of the district means are close for the three learners, while the variance components and the MSEs differ with the learner. The comparison is on the model side only: which learner is *right* for a given application is a question for cross-validation on held-out units (`tune_learner()`, next section) and for simulation.

```r
cmp <- data.frame(district = m_rf$Indicators$district, ranger = m_rf$Indicators$Mean,
                  xgboost = m_xgb$Indicators$Mean, svm = m_svm$Indicators$Mean)
round(cor(cmp[, -1]), 3)
#>         ranger xgboost   svm
#> ranger   1.000   0.988 0.995
#> xgboost  0.988   1.000 0.987
#> svm      0.995   0.987 1.000
rmse_pct <- function(m) 100 * sqrt(m$MSE_Estimates$Mean) / m$Indicators$Mean
data.frame(learner = c("ranger", "xgboost", "svm"),
           sigma_u = round(c(m_rf$MERFmodel$RanEffSD, m_xgb$MERFmodel$RanEffSD, m_svm
           sigma_e = round(c(m_rf$MERFmodel$ErrorSD, m_xgb$MERFmodel$ErrorSD, m_svm$M
           median_RMSE_pct = round(c(median(rmse_pct(m_rf)), median(rmse_pct(m_xgb))
           seconds = round(as.numeric(c(t_rf, t_xgb, t_svm))))
#>   learner sigma_u sigma_e median_RMSE_pct seconds
#> 1  ranger    3298    5534             5.3      14
#> 2 xgboost    2469    5191             5.1      66
#> 3     svm    2548    5559             5.4     123
pairs(cmp[, -1], main = "District means by learner")
```



Totals and aggregates work the same way for every learner: `SAEaggregate()` uses the stored bootstrap replicates.

```
SAEaggregate(m_svm, unique(eusilcA_pop[, c("district", "state")]), name = "state")[,
#>           state n_domains N_units     Mean  Mean_MSE Mean_RMSE
#> 1    Burgenland         9     799 19974.10 908496.14  953.1506
#> 2     Carinthia        10    1723 19988.82 500932.19  707.7656
#> 3 Lower Austria        24    4619 19561.81  37763.36  194.3280
#> 4      Salzburg         6    1671 20352.28 398580.77  631.3325
#> 5        Styria        13    3386 19286.62  66772.33  258.4034
#> 6         Tyrol         9    1889 18550.73  71613.20  267.6064
#> 7 Upper Austria        18    4071 20160.46  51869.13  227.7480
#> 8        Vienna         1    5857 19948.07 206457.63  454.3761
#> 9    Vorarlberg         4     985 22190.49  91476.15  302.4502
```

4. Choosing and tuning a learner with spatial cross-validation

`tune_learner()` fits the mixed model with the learner for every row of a parameter grid and every fold, and summarises the accuracy of the held-out units (on the original scale when a Box-Cox transformation is used). Folds should respect the spatial structure of the sample: `spatial_folds()` builds them from square blocks. The example data have no coordinates, so random ones are drawn here only to show the mechanics.

```
coords <- cbind(runif(nrow(smp), 0, 100), runif(nrow(smp), 0, 100))
folds <- spatial_folds(coords[, 1], coords[, 2], block_size = 10, k = 4, seed = 2)
table(folds)
#> folds
#>   1   2   3   4
#> 486 487 486 486
tn <- tune_learner(Y = smp$eqIncome, X = X, dName = "district", smp_data = smp, learn
                   grid = expand.grid(nrounds = c(100, 200), max_depth = c(3, 5)), fo
                   cores = 2, seed = 3, oof_folds = 3, eta = 0.05)
tn$results[, c("nrounds", "max_depth", "RMSE", "RMSE_se", "bias", "slope")]
```

```
#>   nrounds max_depth     RMSE  RMSE_se      bias     slope
#> 1     100         3 5664.927 265.2620 127.00381 1.0041977
#> 2     200         3 5411.828 264.4835 110.87704 1.0071454
#> 3     100         5 5483.488 275.0904 131.26073 0.9828985
#> 4     200         5 5331.766 281.0964  88.59415 0.9983267
tn$best[, c("nrounds", "max_depth")]
#>   nrounds max_depth
#> 4     200         5
```

The same function with `learner_fun = learner_ranger` and a grid over `mtry` and `min.node.size` tunes the forest, and with `transform = "boxcox"` and `bc_lambda` the comparison is made on the original scale of a transformed model.

5. Box-Cox transformation with any learner

The transformation is independent of the learner: the model is fitted to `((y + shift)^lambda - 1) / lambda`, and the domain means are the expected back-transformed values given the predicted random effects (Gauss–Hermite quadrature on the transformed scale). Learners whose predictions are not bounded by the training range (`bounded = FALSE`, e.g. linear models) get wider back-transformation tables automatically.

```
smp$ypos <- smp$eqIncome - min(smp$eqIncome) + 1e5    # a positive target for the ex
m_bc <- SAEforest_model(Y = smp$ypos, X = X, dName = "district", smp_data = smp, pop_
                        MSE = "none", seed = 1, cores = 2, verbose = FALSE, learner =
                        transform = "boxcox", bc_lambda = 0.5, bc_shift = 1)
#> Warning in SAEforest_mean_scalable(Y = Y, X = X, dName = dName, smp_data =
#> smp_data, : 1 population units outside the back-transformation grid (clamped).
head(m_bc$Indicators, 3)
#>    district     Mean
#> 1 Amstetten 114337.6
#> 2     Baden 122077.0
#> 3   Bludenz 112545.9
```

References

Battese, G. E., Harter, R. M., & Fuller, W. A. (1988). An error-components model for prediction of county crop areas using survey and satellite data. *Journal of the American Statistical Association*, 83, 28–36.

Hajjem, A., Bellavance, F., & Larocque, D. (2014). Mixed-effects random forest for clustered data. *Journal of Statistical Computation and Simulation*, 84(6), 1313–1328.

Krennmair, P., & Schmid, T. (2022). Flexible domain prediction using mixed effects random forests. *Journal of the Royal Statistical Society: Series C*, 71(5), 1865–1894.

Mendez, G., & Lohr, S. (2011). Estimating residual variance in random forest regression. *Computational Statistics & Data Analysis*, 55(11), 2937–2950.

Molina, I., & Rao, J. N. K. (2010). Small area estimation of poverty indicators. *Canadian Journal of Statistics*, 38(3), 369–385.